



Einsendeaufgaben zum Modul 63611 "Einführung in die objektorientierte Programmierung"

Lektion 01

Bitte beachten Sie: Die Einsendeaufgaben sind bei Modul 63611 nicht ohne weiteres mit den Klausuraufgaben gleichzusetzen. In der Klausur kommen auch Aufgaben vor, zu deren Lösung Sie Begriffe und/oder konzeptionelle Zusammenhänge erklären müssen. Diese können den Einsendeaufgaben nicht sinnvoll abgefragt werden, weil man die Antworten einfach im Lehrtext nachlesen kann.

Das Lehrgebiet stellt Ihnen aber die Klausuren der letzten Semester incl. Musterlösungen zur Verfügung, damit Sie einen realistischen Eindruck von der Klausur bekommen können. Sie finden diese in der Moodle-Umgebung des Moduls unter "Arbeitsmaterial".

1. Die Abgabe erfolgt ausschließlich über das Online-Übungssystem.
2. Als Beleger bzw. Belegerin des Moduls 63611 können Sie mit Ihrem FernUni-Account Ihre Lösungen dort einstellen.
3. Eine Handkorrektur Ihrer Einsendung erfolgt nicht automatisch, sondern nur auf Anforderung. Weitere Informationen dazu finden Sie im Moodle des Moduls unter "Einsendeaufgaben".
4. Sobald eine Korrektur der Aufgaben für Sie bereitsteht, werden Sie per E-Mail informiert.
5. Sollten Sie in Sonderfällen das Online-Übungssystem nicht nutzen können (z.B. wegen körperlicher Einschränkungen oder als JVA-Insasse), nehmen Sie bitte telefonisch (02331 - 987 - 2998) oder per E-Mail (sekretariat.steimann@fernuni-hagen.de) Kontakt mit unserem Sekretariat auf.

Aufgabe 2: Fallunterscheidungen**(20 Punkte)**

Der Dortmunder Tierpark hat beschlossen, ein Programm schreiben zu lassen, das Besuchern Auskünfte über die im Zoo vorhandenen Tiere geben kann. Die Implementierung übernimmt das Startup HagenSoft, welches aus Informatik-Studierenden der FernUniversität besteht, die gerade ihre ersten Java-Kenntnisse erworben haben. In der ersten Ausbaustufe gibt ihre Implementierung zu einem Tier lediglich einige elementare Eigenschaften aus. Wird das Programm mit dem Parameter „Spinne“ aufgerufen, lautet die Ausgabe

```
8 Beine  
flugfähig: nein  
hat Federn: nein  
gefährlich für Menschen: eventuell
```

In seiner Implementierung benutzt das HagenSoft-Team `if-else`-Anweisungen zur Fallunterscheidung. Um zu prüfen, ob zwei Strings gleich sind, wird die Methode `equals()` verwendet. Diese ruft man auf einem Ausdruck (z.B. einer Variablen) vom Typ `String` auf. Das entspricht in der OOP-Welt dem Versand einer Nachricht an das vom Ausdruck referenzierte `String`-Objekt. Dabei wird als Parameter eine Referenz auf einen zweiten `String` übergeben. Die Methode liefert dann als Rückgabewert den Wert `true`, wenn die Strings die gleiche Zeichenkette repräsentieren und den Wert `false`, wenn sie sich unterscheiden.

- a) Schreiben Sie ein solches Programm, wie es das HagenSoft-Team getan hat. Es sollte für insgesamt 10 verschiedene Tiere Auskunft über die obigen vier Eigenschaften liefern können. **(15 Punkte)**
Achtung: Bei dieser Aufgabe geht es noch nicht um objektorientierte Programmierung, sondern um Java-Grundlagen. Lösen Sie die Aufgabe daher rein imperativ mit nur einer Klasse, in deren `main`-Methode sich das komplette Programm befindet.
- b) Halten Sie das Vorgehen für praxistauglich? Wo sehen Sie Fehlerquellen? **(5 Punkte)**
(Eine Lösung zur Vermeidung dieser Fehlerquellen brauchen Sie nicht benennen. Diese werden Sie später im Kurs kennenlernen. Wenn Sie möchten, können Sie natürlich auch jetzt schon darüber nachdenken.)

Aufgabe 3: Wundersame Zahlen**(20 Punkte)**

In der folgenden Aufgabe geht es um die „wundersamen Zahlen“. Bekannt geworden sind diese wohl vor allem durch das Buch „Gödel, Escher, Bach – ein Endloses Geflochtenes Band“ von Douglas R. Hofstadter. In der Mathematik sind diese Zahlen im Kontext des sog. Collatz-Problems (<https://de.wikipedia.org/wiki/Collatz-Problem>) bekannt.

Gegeben sei folgender Algorithmus:

Eine Ausgangszahl wird, falls sie gerade ist, durch 2 dividiert. Wenn sie hingegen ungerade ist, wird sie mit 3 multipliziert und dann 1 dazuaddiert. Dieses wird wiederholt, bis das Ergebnis 1 ist. Tritt dieser Fall irgendwann ein, nennt man die Ausgangszahl wundersam. Man geht zwar davon aus, dass *alle* natürlichen Zahlen wundersam sind, allerdings gibt es dafür bisher noch keinen endgültigen Beweis.

Interessant ist neben der Frage, ob eine Zahl wundersam ist, auch die Anzahl der Schritte, die ggf. benötigt werden, bis man bei 1 ankommt. Nennen wir die Funktion, welche eine wundersame Zahl n auf die Anzahl der Schritte abbildet $wunder(n)$. Dann gilt z.B.:

$wunder(5) = 5$ (mit den Schritten 16, 8, 4, 2, 1)

$wunder(22) = 15$ (mit den Schritten 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1)

$wunder(1) = 0$ (keine Schritte nötig, weil bereits der Ausgangswert 1 ist)

Achtung: Auch bei dieser Aufgabe geht es noch nicht um objektorientierte Programmierung, sondern zunächst um Java-Grundlagen, weswegen hier auch noch keine Objekte erzeugt werden, sondern ausschließlich Klassenmethoden verwendet werden. Klassenmethoden gehören nicht zu Objekten, sondern zur Klasse selbst. Man erkennt sie an der Verwendung des Schlüsselwortes `static` (wie bei der `main`-Methode). Aufgerufen werden Klassenmethoden auf dem Klassennamen, also z.B. `Math.sin(42)`. Klassenmethoden entsprechen in mancherlei Hinsicht den globalen Funktionen aus der prozeduralen Programmierung. Wenn Sie also das Gefühl haben, dass Sie Ihre Lösung in sehr ähnlicher Form auch in einer prozeduralen Sprache wie Pascal hätten schreiben können, ist das völlig in Ordnung.

- a) Schreiben Sie eine Klasse `WunderTest` mit einer Klassenmethode `wunder()`, der man beim Aufruf einen `int`-Wert n übergibt und die iterativ (in einer Schleife) den Wert von $wunder(n)$ berechnet und zurückgibt. Eine Rückgabe der Zwischenwerte ist *nicht* erforderlich, es reicht die Anzahl der Schritte. Wenn Sie die Zwischenwerte auch sehen möchten, können Sie eine entsprechende Ausgabe *in* die Methode einfügen. **(8 Punkte)**
- b) Schreiben Sie eine `main`-Methode, welche für alle n von 1 bis 100 durch Aufruf der Klassenmethode `wunder()` den jeweiligen Wert von $wunder(n)$ berechnet und am Ende ausgibt, welche Zahl die meisten Schritte benötigte und wie viele das waren. **(4 Punkte)**
- c) Ergänzen Sie die Klasse `WunderTest` durch eine Klassenmethode `wunderRek()`, die dasselbe leistet wie `wunder()`. In dieser Methode soll allerdings keine explizite Schleife vorkommen. Stattdessen soll sie das Ergebnis durch rekursiven Aufruf ihrer selbst berechnen. Eine Rückgabe der Zwischenwerte ist *nicht* erforderlich, es reicht die Anzahl der Schritte. Wenn Sie die Zwischenwerte auch sehen möchten, können Sie eine entsprechende Ausgabe *in* die Methode einfügen. **(8 Punkte)**

- d)** In Java ist die Idee, die Berechnung der Schrittzahl rekursiv durchzuführen, etwas, worüber man gut nachdenken sollte. Haben Sie eine Idee, wo dabei ein Problem liegen könnte?

(Bonus)

Aufgabe 4: Game of Life**(20 Punkte)**

Das Spiel *Game of Life* ist ein Ende der 60er Jahre vom Mathematiker John Horton Conway erstmalig vorgeschlagener und detailliert untersuchter zellulärer Automat. Gegeben ist dabei ein rechteckiges Feld mit n Zeilen und m Spalten, dessen Komponenten Zellen genannt werden. Jede Zelle kann zwei Zustände annehmen: lebend oder tot. Die zu Beginn lebenden Zellen bilden die Anfangsgeneration. Eine Folgegeneration entsteht nach folgenden Regeln:

- Eine lebende Zelle lebt auch in der Folgegeneration, wenn genau zwei oder drei der acht benachbarten Zellen in der aktuellen Generation leben.
- Eine lebende Zelle stirbt in der Folgegeneration an Einsamkeit oder Überbevölkerung, wenn in der aktuellen Generation weniger als zwei oder mehr als drei der acht Nachbarzellen leben.
- Eine tote Zelle wird in der Folgegeneration lebendig, wenn genau drei ihrer acht Nachbarn in der aktuellen Generation lebendig sind.
- Unabhängig von den obigen drei Regeln sind die Randzellen (also diejenigen Zellen, die nicht genau acht Nachbarn haben) immer tot.

Unten stehendes Programmfragment implementiert einen Teil der Funktionalität des Game of Life. Die aktuellen Zustände der Zellen sind in der zweidimensionalen Array-Variable `feld` gespeichert, der Wahrheitswert `true` steht dabei für eine lebende Zelle, der Wert `false` für eine tote Zelle. Die Methode `print()` gibt die aktuelle Spielsituation auf der Konsole aus. Ihre Aufgabe ist es, die Methode `nextGeneration()` derart zu implementieren, dass `feld` nach Aufruf der Methode die Zellzustände der Folgegeneration entsprechend der obigen vier Regeln beinhaltet.

Hinweise:

- Da Sie für die Berechnung aller Zellen der Folgegeneration noch Zugriff auf die Zustände der aktuellen Generation benötigen, sollten Sie die Zustände der Folgegeneration zunächst in ein neues, zweidimensionales `boolean`-Array gleicher Größe schreiben.
- Ihre Implementierung sollte nach Möglichkeit unabhängig von der Größe der Zellkultur funktionieren; Sie dürfen aber von einer rechteckigen Zellkultur ausgehen.

Prüfen Sie anschließend durch Aufruf der `main`-Methode, wie sich die vorgegebene Kultur über 10 Generationen hinweg entwickelt.

```
public class GameOfLife {
    boolean[][] feld = {
        { false, false, false, false, false },
        { false, false, true, false, false },
        { false, false, true, false, false },
        { false, false, true, false, false },
        { false, false, false, false, false } };

    void print() {
        for (int i = 0; i < feld.length; i++) {
            for (int j = 0; j < feld[i].length; j++) {
                if (feld[i][j]) {
                    System.out.print("o ");
                } else {
                    System.out.print(". ");
                }
            }
            System.out.println();
        }
    }

    void nextGeneration() {
        /* ... */
    }

    public static void main(String[] args) {
        GameOfLife myGame = new GameOfLife();
        for (int i = 0; i < 10; i++) {
            myGame.nextGeneration();
            myGame.print();
            System.out.println();
        }
    }
}
```

Aufgabe 5: Ausnahmebehandlung**(20 Punkte)**

- a) Wie lautet der Satz, der von dem unten angegebenen Programm bei Ausführung ausgegeben wird? **(5 Punkte)**
- b) Beschreiben Sie detailliert, was bei der Ausführung des Programms passiert, insbesondere, welche Exceptions auftreten und wo diese abgefangen und behandelt werden! **(15 Punkte)**

```
01 public class ExceptionTest {
02     public static void main(String[] args) {
03         int zahl = 13;
04         try {
05             try {
06                 int[] b = { 3, 11, 4, 9 };
07                 for (int i = 0; i <= b.length; i++) {
08                     if (b[i] % 3 == 0) {
09                         zahl = zahl - 1;
10                     }
11                 }
12                 System.out.println("Eine");
13                 zahl = zahl - 4;
14             } catch (ArithmeticException e) {
15                 zahl = zahl + (true & false ? 3 : 2);
16                 System.out.println("Ausnahme");
17             } catch (Exception e) {
18                 zahl = zahl + (zahl == 14 ? 3 : 2);
19                 System.out.println("Ausnahmen");
20             } finally {
21                 zahl = zahl + 4 / 7;
22                 System.out.println("sind");
23             }
24             switch (zahl) {
25                 case 5:
26                     System.out.println("eine");
27                     break;
28                 case 7:
29                     System.out.println("nicht");
30                     break;
31                 case 14:
32                     System.out.println("macht");
33                     break;
34                 case 13:
35                     System.out.println("gar");
36                 case 8:
37                     System.out.println("nicht");
38                     throw new Exception();
39                 default:
40                     System.out.println("keinen");
41                     break;
42             }
43             } catch (Exception e) {
44                 String s = (zahl % 6 / (7 % 6 % 2)) == 1 ? "so" : "Sommer";
45                 System.out.println(s);
46                 return;
47             } finally {
48                 System.out.println("schlimm!");
49             }
50             System.out.println("glücklich");
51         }
52     }
```